

00 At a glance

Markdown does not need a new syntax. It needs a **compiler** — something that can point at a paragraph today and still find that paragraph after an editor, an agent, a formatter and a colleague's `git pull` have each rewritten the file.

That is the whole thesis, and every number in this document was measured rather than assumed. Where the research overturned our own earlier position, the correction is recorded in place.

RESEARCH SCALE

~14M

subagent tokens · ~90 agents · 20 experiments

ANCHOR ACCURACY

99.63%

correct, at 0.050% false match

CORPORA

4,400+

real markdown files, three vaults

NEW FORMAT

None

the file stays ordinary .md

Contents

01	The problem, and why it is not a format problem	02
02	What we researched	03
03	How markdown actually works	04
04	How the industry does it	05
05	What we found	06
06	Taking markdown to the next level	07
07	Defining our own representations	08
08	frontmatter as the first renderer	09
09	Fusing into the AI OS	10
10	The plan and its kill-gates	11
11	Benchmarks, and what is still open	12

01 The problem, and why it is not a format problem

A markdown file is a **string**. Every tool that touches it hands back a *different string* with no record of what changed. So nothing in the file can be referred to twice: a comment thread, a backlink, an outline entry, a cross-file link to a heading — each is a pointer into text that will move, and nothing can say where it went.

Every serious document system solved this. Each solved it using a property we do not have.

SYSTEM	ANCHOR	SURVIVES OUTSIDE EDIT	WHY IT WORKS FOR THEM
Word	zero-width sentinel pairs	yes	the artifact is XML; an invisible marker is free
Notion	a UUID on the block	yes	the artifact is a database
Overleaf	offsets in an OT sidecar	no	they own <i>every</i> write to the file
ProseMirror / CM6	offsets through a change map	no	the remap needs the op that caused the change

Overleaf is the only published system with our exact constraint — the artifact must stay a plain text file another tool can read — and it survives only by owning every mutation. **We cannot.** A `git pull` yields a new string with no op stream. A `vim` edit yields no op at all.

So the thing to invent was never a syntax. It is an anchor that survives the git boundary.

The starting point. This began as `mdz` — a brief for a next-generation markdown. Measurement moved the conclusion twice. The final position is narrower, and far more defensible: the format is fine, the tooling is missing.

02 What we researched

Roughly **14M subagent tokens across ~90 research agents**, ~20 local experiments and a six-area utilization sweep, against three real corpora: `md` (4,117–4,244 files), `knowledge` (272–410) and the product repo (52).

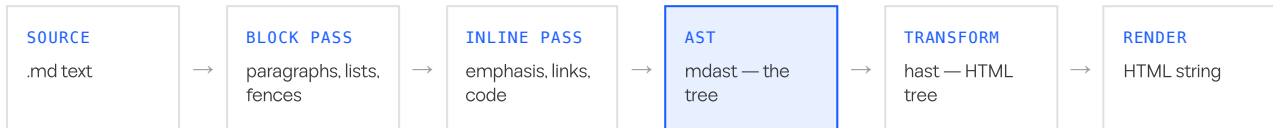
Every headline number was re-derived in the main loop before being written down. Several agent claims failed that check and were corrected rather than quietly fixed — including two of our own.

AREA	QUESTION	METHOD
Anchoring	can a block keep identity across edits?	384-config sweep, 41,642 block-versions, 294 real revision pairs, every error hand-audited
Round-tripping	can source be regenerated from an AST?	CommonMark spec analysis + full-corpus round trip
Carrier	where does an anchor physically live?	1,765 blocks × 6 carrier designs
Container	can many files live inside one <code>.md</code> ?	7 delimiter designs × round-trip survival
Extensibility	can markdown absorb new constructs?	5 agents briefed to argue <i>for</i> extension, against our own prior
Notation	can a model read a declared syntax?	4 notation arms × read / write / edit-under-load, 1,200 judgments
Demand	what do people actually want?	6,047-plugin catalogue, 132.8M downloads, forum + VS Code cross-checks
Constructs	what is markdown actually made of?	frequency census over 42,643 files
Security	what breaks on hostile input?	76 payloads × 9 renderers = 684 DOM-verified runs
Accessibility	what can an automated checker see?	axe-core over 2,286 rendered files
i18n	what breaks outside English?	4 engines × CJK, Bengali, Arabic, Hebrew, Thai, Khmer

Method note. Five findings overturned positions this project had already written down as settled — including "no new extension mechanism", "rigid context fingerprints", and "markdown merges worse than code". A plan that never contradicts itself was never tested.

03 How markdown actually works

The pipeline is what makes the design decisions obvious rather than arbitrary.



Markdown parses in **two passes** — block structure first, then inline markup inside each block. That is why `bold` cannot span a paragraph break, and why a fence's contents are invisible to every inline rule. **The arrow into the AST is one-way.** Everything after it is a projection.

The AST is lossy, and it is provable

Foster et al. (TOPLAS 2007, Lemma 3.9) prove a total well-behaved lens whose `put` ignores the original source requires `get` to be a bijection. "Regenerate from the AST" is definitionally such a lens. What that costs, measured:

MEASUREMENT	RESULT
CommonMark spec examples that are two <i>distinct sources</i> with an identical AST	443 of 655
Lines rewritten by a zero-edit round trip of a real 206 KB document	24.98%
Bytes dropped by that same round trip	18.94%
Files surviving <code>parse → stringify</code> byte-identically	0 of 51

`mdast-util-to-markdown`'s own documentation says it plainly: "*complete roundtripping is impossible.*" Hence the first invariant — **splice, never regenerate.**

And the tree is closed. A `.md` file carries a finite ordered tree over a **closed 21-label alphabet** with string leaves. It cannot carry types, identity, null, or **any edge that is not parent-child**. That is the tree/graph boundary — not a syntax gap a dialect could close.

04 How the industry does it

APPROACH	WHO	MECHANISM	OUTCOME
Superset <i>with</i> a runtime	MDX	JSX inside markdown, compiles to JS	3.07% of <code>.md</code> volume. Cannot write the document back out.
Superset, no runtime	Markdoc (Stripe)	<code>{% tag %}</code>	0.136%. Byte-identical <code>format()</code> .
Clean-slate redesign	djot (jgm)	a new grammar fixing markdown's ambiguities	693/wk vs remark's 45.7M — a 65,990× gap
Directive plugin	remark-directive	<code>:::name</code> generic blocks	6.95% of remark — the only extension at scale
Fence dispatch	mermaid, and ~everyone	match the info string, replace the node	30 platforms, zero coordination
Sidecar file	Obsidian	<code>.canvas</code> / <code>.base</code>	shipped twice, for the two hardest gaps
Compile to a lock file	GitHub Agentic Workflows	<code>workflow.md</code> → <code>.lock.yml</code>	in production, 2026

Three lessons, pointing the same way

New dialects do not win. djot has CommonMark's own author and is invisible. The reason is not quality — markdown's value *is* its universality, and a dialect forfeits exactly that.

Fence dispatch wins because it costs nothing. CommonMark, verbatim: *"Although this spec doesn't mandate any particular treatment of the info string, the first word is typically used to specify the language."* That refusal to define semantics *is* the extension point. A renderer matches one string and replaces the node; every non-participating tool degrades to a code block.

And the sobering one. Faced with its own largest format gap — Dataview, 4.65M downloads — Obsidian did not extend markdown. It shipped `.base` (YAML) and `.canvas` (JSON, whose `edges` are directed, labelled and typed). The two things markdown provably cannot express each got a non-markdown sidecar. The best-resourced actor in the ecosystem concluded markdown was the wrong place. Section 08 answers that.

05 What we found

The anchor works

SCHEME	CORRECT	FALSE MATCH	SAFE REFUSAL
byte offset	36.93%	62.12%	0.95%
block index	44.43%	55.50%	0.08%
content hash alone	83.36%	0.00%	16.64%
rigid context fingerprint (k=1/2/3)	90.7 / 88.7 / 87.2%	0.25%	9–13%
ours — anchorable blocks only	99.627%	0.050%	0.323%

~41 ms for a 446 KB, 2,225-block document. All 14 false matches were **hand-audited, not sampled**: 4 oracle artifacts, 6 a bulk migration, 3 list drift, 1 a table rewrite. Genuine substantive errors: **10 in 28,170 = 0.036%**. Three results overturned our earlier design — rigid fingerprints get *worse* as the window grows, chunking is the wrong granularity, and **every position-based tiebreak made things dramatically worse** (0.00% → 22.20% false).

Refusing is cheap. A silently wrong anchor is not.

What the demand data says

6,047 Obsidian plugins, 132,834,339 downloads. Classifying the top 300 — 83.9% of all demand — by *what kind of gap* each fills: **FORMAT 40.1%**, app 31.0%, external 19.5%, render/ergonomics 9.4%. Two-fifths of revealed demand is markdown failing to express something, and the app column is *smaller* — the opposite of "markdown is fine, the apps are limited." It replicates on three populations sharing no mechanism: plugins 40.1%, forum requests 39.0%, VS Code 35.4%.

The causal proof is Logseq. Its format natively has block IDs, properties and a query language. Obsidian has **70 spaced-repetition plugins; Logseq has 2**, and both are merely Anki bridges. Query 46 vs 20. Tasks 149 vs 30. Kanban — which neither format helps — 63 vs 6. Logseq is a *worse app*, yet every cluster matching a native format feature is **10–35× smaller**. When the format absorbs a capability, the plugin ecosystem for it collapses.

The wedge. Obsidian's `processFrontMatter` strips quoting, **deletes YAML comments outright** and destroys type tags. Their own type declaration is `{ [key: string]: any }` — every property in every note, in the market leader, is `any`. Demand ratio 2,300×. They have publicly declined to fix it.

06 Taking markdown to the next level

The answer to “*why can't we have folder trees, self-including files, new representations?*” is **we can** — through dispatch, never through new syntax. Four channels exist.

TIER	CHANNEL	A DUMB VIEWER SEES	PROVEN BY
1	frontmatter	nothing	universal — TOML is 0 of 42,643 files; YAML is the only vocabulary
2	fence info string	a normal code block	30 uncoordinated platforms render mermaid
3	HTML comment	nothing	nodejs/node ships 4,310 <code><!-- YAML</code> instances
4	derived, never stored	—	graphs, outlines, site maps

What must never happen: spending a sigil

Markdown has ~32 punctuation extension points. From jgm: “*two extensions may want to interpret the same character for two different purposes.*” That is **character-namespace exhaustion** — resource exhaustion, not governance, and it is live today: Spaced Repetition's card separator is `:::`, Dataview's inline field is `key:: value`. Two top-50 plugins, same two characters.

What should fuse, ranked by measured demand

PRIMITIVE	INSTALL WEIGHT	OUR CHANNEL
embedded freehand graphics	8.79M	fence + opaque payload
task semantics beyond a checkbox	8.54M	frontmatter vocabulary
a queryable relation over documents	7.22M	resolver, not syntax
typed links	820 likes · #1	resolver + manifest
multi-file composition	449 + 174 · unbuilt	container + transclusion

The last two matter most because they are **supply-starved** — the signature of a gap a plugin *cannot* close. A typed link needs *the link itself* to carry the relation. Templating (7.16M) is deliberately absent: that one is a runtime, and a runtime is what makes a format unreadable.

The MDX lesson is a warning, not a template. MDX **cannot write the document back out** — arbitrary JavaScript does not losslessly re-serialize, which for a product that edits and saves is disqualifying. Take MDX's ambition; refuse MDX's mechanism.

07 Defining our own representations

The proposal was a paired marker — something like ``X` ... `X`` — meaning "this is an image", or a table, with only the frontmatter knowing what it means. **The instinct is right. One detail decides whether it works.**

WHAT FAILS

Inventing a new delimiter in prose. It spends a non-renewable resource, collides with the ~8 incompatible dialects already in the wild, and — critically — **fails silently**. The model writes `~ doing` instead of `~doing`, the renderer does not match, and the field vanishes. The document still looks like a document. Nothing throws.

WHAT WORKS

Declaring a vocabulary inside a construct that is already opaque. A fence and a code span are *inert everywhere* — every markdown tool on earth already renders them literally and refuses to interpret their contents. Put the notation there and degradation is guaranteed by the spec, not by luck.

```
---
fm:vocab:
  due:                                     # carried by an inline code span
  examples: ["`due:2026-08-01`", "`due:friday`"]
  figure:                                 # carried by a fence
  examples: ["```fm:figure\nsrc: a.png\ncap: Fig 1\n```"]
---

Ship the migration `due:2026-08-01` before review.
```

Open that file anywhere else and you see inline code and a code block — nothing breaks, nothing is lost, no tool errors. Open it in frontmatter and the compiler binds `due` to a date type, checks it, indexes it, and projects it into the graph.

Three rules govern this, all measured

RULE

EVIDENCE

- | | |
|---|--|
| 1 · Declare by EXAMPLE , never by rule | Aycock et al. ablated MTOB: "almost all improvements stem from the book's parallel examples rather than its grammatical explanations" — and flatly, "no evidence that long-context LLMs can make effective use of grammatical explanations." |
| 2 · Cap the vocabulary in single digits | IFScale, 20 models: "even the best frontier models only achieve 68% accuracy at the max density of 500 instructions." Plus primacy bias — the declaration sits where compliance is highest, the constructs fire where it is lowest. |
| 3 · The renderer is the write gate | Constrained decoding coverage on complex schemas: Guidance 41%, llama.cpp 39%, XGrammar 28%, Outlines 3%. Validate-and-repair gets the same guarantee at no distribution cost. |

The measurement that decides the operating point has not been made. Every number above is Opus-class, five constructs, declaration in immediate context. The model that will actually write into a user's file is smaller and faster, and every published curve says that is precisely where format adherence collapses. **This establishes a ceiling, not an operating point.**

08 frontmatter as the first renderer

The honest answer is better than lock-in: **exclusivity comes from capability, not captivity**. The files stay portable on purpose — that is what makes them safe to adopt, and safety is the acquisition mechanism.

THE SAME BYTES, IN ANY OTHER TOOL

Renders as ordinary markdown. The notation is inert text.
Nothing breaks, nothing is lost, no tool errors.

THE SAME BYTES, IN FRONTMATTER

The compiler reads the declaration, binds and type-checks the constructs, keeps block identity across every edit, and projects graph, outline and site.

Why the capability gap is durable

PROPERTY

WHY NOBODY ELSE HAS IT

Identity is the moat

A sidecar can add expressiveness — Obsidian proved it twice. A sidecar **cannot** give identity across an out-of-band edit. That needs the compiler.

The editor sees the save

Drift detection without a diff is near-unsolved — a trivial heuristic scores F168, beating every post-hoc neural model at 66–67. Handed the edit, neural reaches 77–81.

The wedge is publicly refused

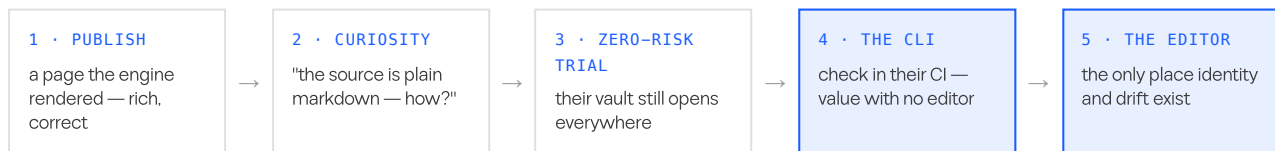
Competitors do not accidentally close a gap they have declared they do not want to close.

Markdown-aware diagnostics

axe-core fires 7 of 105 rules on markdown; 98.5% of violations are page chrome. We found `remark-math` swallowing prose between two `$` signs in 20 of 272 files — **zero** axe violations.

The acquisition strategy — the mermaid playbook

You cannot force adoption of a portable format, and trying is what kills formats. What you *can* own is being **the reference implementation of something everyone wants**. Mermaid is the proof: the syntax was free, nobody was forced, GitHub adopted it, and 30 platforms followed. The *spec* was the giveaway; the *tooling* was the product.



How to read it. Steps 1–3 are **free and open** — the format degrades, so trying costs nothing and leaving costs nothing. That is what makes people willing to start. Steps 4–5 are where the compiler is *required*, because identity-across-edits and save-time drift cannot be reimplemented by someone merely reading the file. Publishing benchmark B1 is the accelerant.

09 Fusing into the AI OS

This is the part with the least friction, because **the AI OS is already markdown-native** — and currently runs on *unchecked* markdown.

AIOS SURFACE	TODAY	WITH THE ENGINE
<code>SKILL.md</code> × ~200 skills	YAML frontmatter, no schema, no validation	pass 3 types it; <code>frontmatter check</code> gates skill authoring
<code>CLAUDE.md</code> + Learned Rules	append-only prose, rules cited by number	block identity — a rule keeps its anchor when the file is edited
memory <code>*.md</code> + index	<code>[[wikilinks]]</code> resolved by convention	pass 4 resolves them; a dangling link is a diagnostic, not a silent drop
knowledge base, 4,200+ notes	graphify rebuilds a graph out-of-band	pass 5 <i>is</i> the graph builder — incremental, at save time
trace ledger <code>*.jsonl</code>	JSONL rows	engine diagnostics emit as trace rows — the loop gets format signal
snapshot / recall cards	hand-written <code>.md</code>	splice-safe writes; the UTF-8 truncation class disappears

The sharpest fit is a bug class already on the record

Learned Rule #59 records three instances of a producer writing one field name and a consumer reading another — `tokens_in` vs `input_tokens`, `pass` vs `assertion_pass` — each silently publishing a wrong number for weeks, because a missing key coerces to a falsy default that is indistinguishable from a real negative. **That is exactly what a typed interface pass catches at author time.**

The engine enters at the data layer. Never at the routing layer.

Sequencing is already prescribed by the system's own rules. **Rule #43** — build in hierarchy order, so the engine lands on traces and validated skill frontmatter before anything touches routing. **Rule #38** — nothing goes dev→live: offline eval, then shadow, then promote. **Rule #20** — parallel work must operate on disjoint artifacts, which the per-file compiler model satisfies by construction.

What it buys the loop. Today the orchestrator's own substrate — skills, memory, rules, knowledge — is the one part of the system with no type checking, no link resolution and no identity. Every other layer has gates. Applying the compiler to `.md` closes the last unchecked surface, and it does so with a CLI that needs no editor and no product decision.

10 The plan and its kill-gates

```

1 parse      .md → AST + byte positions    incremental
2 identify   mint / recover block anchors  ← the invention
3 interface  frontmatter → module signature schema INFERRED, never demanded
4 resolve    bind every cross-reference    unresolved = diagnostic, never a silent drop
5 index      symbol table + reverse index   → outline, tags, backlinks
6 check      schema · links · drift         → the confidence ladder
7 project    outline · graph · site · tokens never stored

```

Each phase carries a gate that can **stop** it. A phase without a falsifiable gate is a wish.

PHASE	SHIPS	KILL-GATE
1 · core/	splice writer, anchors, the YAML wedge	100% byte-fidelity on untouched regions, 50 files, verified by an independent oracle — the current check uses the writer's own code path and cannot catch an offset bug
2 · resolve/check/	linker, symbol table, frontmatter check	per-class precision across 4,117 files
3 · schema/	infer → propose → accept	false-positive rate on a corpus <i>verified human-authored</i> ; above ~0.4 users disable it permanently
4 · pack/unpack	container + edit-mapping back to source	byte-identical unpack, per-file digests
5 · reconciler	prose ↔ code drift	tier I only at first

Two rules keep it usable

A diagnostic may only reach tier E — the tier that fails a build — if a human accepted the constraint that produced it. Inferred schemas start at Warn; accepting the proposal is what converts advice into a guarantee. A wrong inference can never break a build.

And a scope check from our own data. A compiler run over 833 authored files found 365 true problems touching only **112 files (13.4%)**, and one file held 56% of everything. A random note therefore has an **86.6% chance of an empty panel**. A vault-level **check** command is earned by that data; a persistent Problems panel is not.

11 Benchmarks, and what is still open

Pass the benchmarks that exist. **Publish the ones that do not** — that is how a compiler becomes an industry reference instead of a product feature.

LAYER	TARGET
CommonMark 0.31.2 + GFM suites	100%, non-negotiable
Differential fuzz vs <code>cmark-gfm</code>	zero unexplained divergences
DoS corpus — <code>remark-parse</code> is quadratic and crashes on 20 KB of nesting ; markdown-it does it in 12 ms	depth cap + wall-clock budget, enforced in a worker
B1 · anchor durability — six baselines already measured against ours	the moat play. Publish with a runner and a leaderboard.
B2 · splice fidelity — byte-identity outside the edited span	100%, permanent CI gate
B6 · renderer security — already built and live	55 payloads, DOM-verified, fails 28/55 against an unprotected pipeline

The rule governing every benchmark. A benchmark whose subject is a rare fault proves nothing until it reproduces the fault. A green suite is the expected result of running it, not evidence of correctness. Every gate must be shown to *fail* against the unfixed implementation before its pass is reported.

Open — and honest

The notation result is **Opus-class only**; the production-model measurement is unmade. The schema-inference corpus was **voided** by its own verifier (27.2% vendored, 24.4% LLM output). Multi-block moves are **simulated**; the merge path is **untested**. Three decisions are open: the CRDT architecture conflict, whether the notation ships at all, and ten verified bugs in shipped editor code.

The risk, recorded rather than buried. Obsidian, holding every advantage, looked at these same gaps and shipped sidecar files twice. Our differentiator is **identity across edits** — which a sidecar cannot provide — not expressiveness, which a sidecar provides perfectly well. If we ever compete on expressiveness we have lost, because `.canvas` already won that fight.